DEVOPS & CLOUD [SWE] · SCALER ACADEMY

$ cat notes.md

DevOps & Cloud [SWE] — Class 2 Notes

LECTURE TITLE	**Linux Fundamentals**
DATE/TIME	**13 August 2026, 2:00 PM**
DURATION	**120 minutes**
INSTRUCTOR	**Nensi Ravaliya**
BIG IDEA	**Everything in Linux is a file — so once you can create, read, move, permission and kill files, you can run a server.**

WHAT'S INSIDE — 25 SECTIONS

00 ⚡ The 60-Second Version
01 🗓️ Housekeeping & Setup
02 🖥️ Building a Server From Scratch (Live EC2 Demo)
03 🔑 Getting Inside — SSH
04 🔹 The Very First Thing You Do On A New Server
05 🔹 Hostname vs Username
06 🎮 Making Files — Four Ways (Team Quiz)
07 🔹 Making Directories
08 🔹 Getting Around
09 🔹 Copy, Move, Rename, Delete
10 📁 Reading Files
11 📁 Listing & Hidden Files (Her Two Tasks)
12 📣 Processes — Linux's Task Manager
13 🔹 history — And The Time It Saved A Bank
14 🔐 Permissions — chmod Properly
15 🔹 / vs /root — The Question They Love Asking
16 📣 Tour of the Linux Filesystem
17 🔎 grep and the Pipe
18 🔗 systemctl — Controlling Services
19 📝 Homework #3 — User Management (The Real Assignment)
20 📣 Absolute Basics (for anyone brand-new to Linux)
21 📚 Resources She Promised
22 📒 Cheat-Sheet — Every Command From Today
23 📝 Homework Round-Up
24 🗒️ Notes on the Transcript Itself

Rebuilt from the raw class transcript — nothing cut, just organised, de-noised and made readable. `theme: Deep Ocean`

00 ⚡ The 60-Second Version

1. **We built a real server first.** An AWS **EC2** instance → pick an OS (Ubuntu) → pick a size (`t2.small`) → download a **key pair** → SSH in. Ten minutes, and you have a Linux box on the internet.
2. **First two commands on any new server, always:** `sudo apt update` then `sudo apt upgrade -y`.
3. **Linux never has just one answer.** There are 6+ ways to get your username, 5+ ways to get your hostname, 4+ ways to make a file. That's a feature, and it's an interview question.
4. **Permissions are just three numbers.** read = 4, write = 2, execute = 1. Three slots: **owner · group · others**. `chmod 761` = owner everything, group read+write, others execute only.
5. **`/` and `/root` are NOT the same thing.** `/` is the top of the filesystem. `/root` is just the root *user's* home folder. This is asked constantly.
6. **`/var/log` is where a DevOps engineer lives.** All system and application logs land there.
7. **`history` saved a bank.** See §11 — the best story of the class.

◆ ◆ ◆

01 🗒️ Housekeeping & Setup

- The recording opens with pure ASR garbage — *"Undertekster av Amara.org-gemenskapen"* repeated (Norwegian subtitle credits leaking in from the transcription tool). Nothing was said; class actually starts at *"It's audible now? See, I'll share my screen."*
- **Instructor flex of the day:** she's an **AWS Community Builder in AI development**, which gives her free AWS credentials to demo with.

Your own AWS access

Thing	Detail
How to get free AWS	Use your **student ID** (`@yourcollege.com` / college Gmail)
Free-tier length	**6 months** — *"previously it was one year, now they reduced it"*
For the assessment	Scaler will give you credentials to log into the same AWS
From next class	*"You can create your EC2 instance and do the hands-on with me."*

💡 GOOD TO KNOW

Why Linux, first, before anything: *"Linux is the heart of DevOps."* Every single day, in every single task — Docker, Kubernetes, Terraform, Ansible, servers, CI/CD — Linux knowledge is what you actually use. That's why it's Class 2 and not Class 12.

◆ ◆ ◆

02 💻 Building a Server From Scratch (Live EC2 Demo)

She did the entire thing on screen. Here's the click-path.

Step 1 — Launch

```
AWS Console → search "EC2" → EC2 Dashboard → Launch instance
Name: R3
```

Step 2 — Pick the operating system (the AMI)

AMI = Amazon Machine Image. It's just "which OS do you want, and which version."

🧠 PICTURE THIS

Picture this: choosing an AMI is exactly like choosing which OS to install on a fresh laptop. Windows? Which one — 10 or 11? Linux? Which flavour — Ubuntu, Fedora, CentOS?

Linux "flavours" (= distributions):

Distro	Note
Ubuntu	what we used
Amazon Linux	Amazon's own in-house Linux
Debian	Ubuntu's parent
RHEL	Red Hat Enterprise Linux
CentOS	RHEL's community cousin
Fedora	bleeding edge

Ubuntu versions available: 26 (newest), 24 (released 2024). Amazon adds a new one every year, named after the year — *"2024 → 24 version, 2026 → 26 version."* She's most comfortable on **24**.

✅ LOCK THIS IN

Two words for the same thing: Linux **flavour** = Linux **distribution**. Don't get thrown when an interviewer switches vocabulary mid-question.

Step 3 — Pick the instance type (the size)

This is the **T series** — the beginner/testing series:

Type	vCPU	Memory	Price/hour
`t2.nano`	1	**0.5 GiB**	$0.0081
`t2.micro`	1	**1 GiB** (double nano)	↑
`t2.small`	1	**2 GiB**	↑
`t2.medium`		more	↑
`t2.large` → `xlarge` → `2xlarge`		keeps doubling	↑↑

💡 GOOD TO KNOW

When to use which series: T series is for *"beginners learning AWS, test purposes, small tasks — a normal web application where you're not adding much load."* Other series exist (M series, and many more) for specific use cases — e.g. an **e-commerce app during a sale** needs a beefier series.

Look for the **"Free tier eligible"** tag if you're on a student ID. She picked `t2.small`.

Step 4 — Key pair (this is your password)

🧠 PICTURE THIS

Picture this: to get into your server you need two things — its **address** (the public IP) and its **key** (the `.pem` file). The key pair *is* the password. Lose it and you're locked out.

```
Create new key pair
  → give it a name
  → algorithm: RSA
  → format: PEM   ← use this one ("majorly we are using the PEM file")
  → Create key pair → it downloads to your Downloads folder
```

Step 5 — Which traffic to allow (security group)

Allow	Why
SSH	enabled by default — *"how will you log in? To log in you at least need SSH."*
HTTP	serve a website
HTTPS	serve it securely

And the **source** setting:

Setting	Meaning
`0.0.0.0/0` ("Anywhere")	*"from any corner of the world you can reach my server"*
My IP	only from your own machine

> 📌 **NOTE**
>
> She used "Anywhere" for the demo and deferred the real discussion: *"When we learn cloud I'll tell you exactly where you need to choose My IP vs Anywhere."*

Step 6 — Launch

Scroll down → **Launch instance** → done. The instance appears in the **Instances** list with a **public IP**.

◆ ◆ ◆

03 🔑 Getting Inside — SSH

Two commands. That's it.

```
chmod 400 mykey.pem                    # lock the key down first
ssh -i mykey.pem ubuntu@<public-ip>    # -i = "use this identity file"
```

Decoding that second command:

Piece	What it is
`-i mykey.pem`	the key file you downloaded
`ubuntu`	the **username** — this is the default for Ubuntu AMIs. *"Did you see me add a username anywhere? No. That means it's the default: `ubuntu`."*

Piece	What it is
`@<public-ip>`	the server's address

On first connect it asks *"Are you sure you want to continue connecting?"* → type `yes`. *"Of course we created it — why not?"*

⚠️ **WATCH OUT**

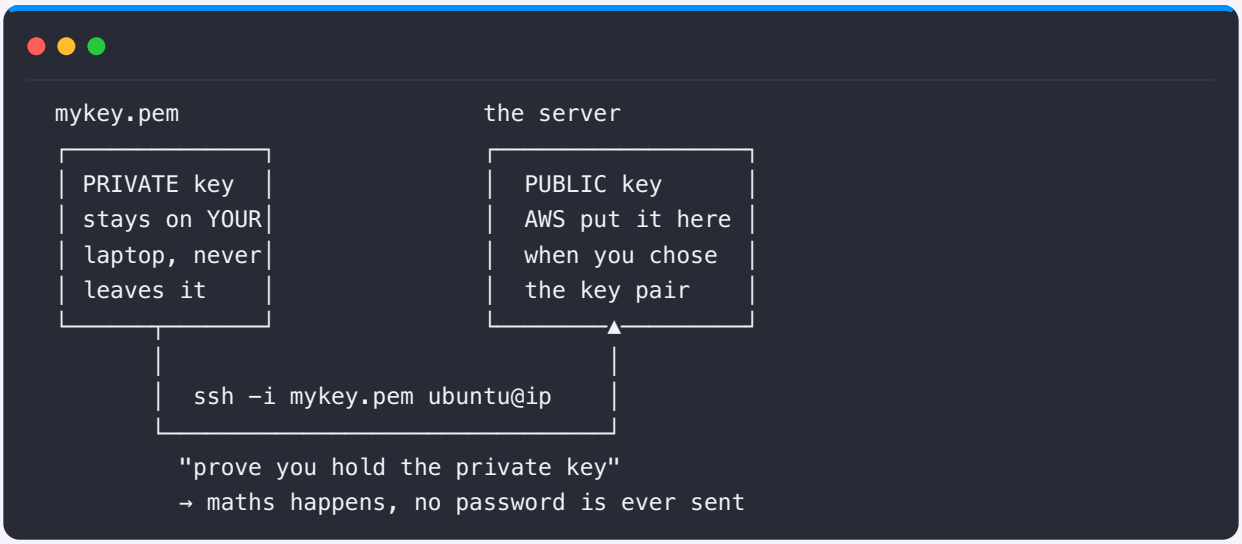
Why `chmod 400` first? `400` means **read-only, for the owner only**. SSH refuses to use a key file that anyone else on the machine could read. (Full permission breakdown in §12.)

🧠 **PICTURE THIS**

What SSH and the key pair actually are — because you just used both without being told.

SSH (Secure Shell) gives you a terminal on a machine somewhere else. You type, it runs there, the output comes back — all encrypted. That's the whole idea.

The key pair is a lock and a key, and the split matters:



✅ **Why this beats a password:** nothing secret crosses the network. The server sends a challenge, your laptop answers it using the private key, and the server checks the answer against the public key. Someone watching the wire learns nothing reusable.

⚠️ **Which means: lose the `.pem` and you are locked out permanently.** AWS cannot recover it — it never had your private key. Back it up.

💡 **And that's why `chmod 400` is enforced.** A private key readable by other users on your machine isn't private any more, so SSH refuses to use it rather than let you shoot yourself.

◆ ◆ ◆

04 🔷 The Very First Thing You Do On A New Server

🎤 SAID IN CLASS

"This is a funny question, but — we just logged into the server. Which command are you executing first?" Someone guessed `whoami`. Nice, but no.

Answer: update it. A fresh server is never up to date.

```
sudo apt update        # refresh the package lists
sudo apt upgrade -y    # actually install the newer versions
```

Bit	Why
`update`	*"It will update all the things — the packages, the drivers, the booting command files, all of it."*
`upgrade`	applies the updates
`-y`	🎯 **the trick question she asked:** without `-y` you get a **y/n prompt** for every package. `-y` says "yes to everything, don't ask me."
`sudo`	*"do this as the superuser"* — see below

💡 GOOD TO KNOW

`sudo` in one line: "run this next command as the administrator."

It stands for **s**uper**u**ser **do**. Installing software touches system folders your normal user can't write to, so `apt` needs it. Reading a file in your own home directory doesn't.

✅ **The habit worth forming: don't prefix everything with `sudo`.** Add it only when a command actually fails with *"permission denied"*. Running as root all the time is how people accidentally delete `/` — and it's the same reasoning behind the `USER` instruction in Class 7.

😄 CLASS MOMENT

"The first batch people were more smart, I guess" — after the room struggled with this one.

◆ ◆ ◆

05 🔷 Hostname vs Username

Look at the prompt you just logged into: `ubuntu@ip-172-31-x-x`

Piece	Which is it?
`ubuntu`	the **username**
everything after `@`	the **hostname**

🧠 PICTURE THIS

Why hostnames exist at all — the DNS alias idea: *"You want to search Google. Do you search `google.com` or do you search `8.8.8.8`?"* Nobody types IPs. A hostname is a friendly **alias** pointing at an IP address. Every device on a network gets one.

📌 NOTE

On a brand-new server, `ifconfig` won't exist — you have to install `net-tools` first. That's normal, not broken.

🔑 The most important idea in the whole class

✅ LOCK THIS IN

Linux always has more than one way. *"If there's a task to get 10 files, or create 10 files with one command — you can. If there's a problem statement, you can solve it with different commands and get the same result. In Linux there is no 'only one solution'."*

This is why she then made the class brainstorm *every* way to do the same three things.

Every way to get the hostname

#	Command	Note
1	`hostname`	the obvious one
2	`hostnamectl`	also gives you **hardware** information
3	`cat /etc/hostname`	read the file directly — *"it must be stored there"* ✅
4	`hostname -s`	short name
5	`hostname -f`	fully-qualified name
6	`hostname -i`	the IP — *"this is a new command, even I don't know it."* 😄

💡 GOOD TO KNOW

"See, it's just a variation. The command is `hostname`, that's it." The flags do the rest.

🎯 Every way to get the username — this is a real fresher interview question

> 🎯 **INTERVIEW ALERT**
>
> **Interview alert:** *"Normally in interviews for freshers they ask: how many ways can you get the username?"* She said there are **seven to eight**. Here's the list the class built:

#	Command	What it gives
1	`whoami`	just the username
2	`who`	who is logged in
3	`w`	same idea, more detail
4	`logname`	login name — *"usually I'm not using this"*
5	**`id`**	⭐ **the most important one** — full identity
6	`echo $USER`	read the environment variable
7	`users`	all logged-in users

What `id` gives you, and what actually matters:

Field	Meaning
UID	user ID
GID	which **group** that user belongs to
groups	all groups
(rest)	sudo-related extras

> ✅ **LOCK THIS IN**
>
> *"We only need UID, GID and groups. That's it."*

Every way to get the OS version

Students tried `neofetch` and `uname` — *"good one, but I want complete information, not just 'Linux'."* One got 80% of the way with `lsb_release`.

✅ LOCK THIS IN

The standard answer she wanted:

```
cat /etc/os-release
```

Gives you the distro (Ubuntu), the version (24), the Debian ID, everything. *"Even if it's an MCQ, they will not give you `lsb_release` as an option — there's a standard way, and this is it."*

Why this matters in real life: if you run **Oracle VirtualBox** with several images installed — Ubuntu, Windows, RHEL, Fedora, CentOS — this is how you check all the versions and details at once.

◆ ◆ ◆

06 🎮 Making Files — Four Ways (Team Quiz)

She turned this into a Team A / B / C relay.

Team	Command	Notes
A	`touch data1`	the fastest way
B	`nano data2`	opens an **editor** — type content, save, exit. Then `cat data2` to see it
C	`echo "hello world" > d3`	writes content *and* creates the file
bonus	`vim d4` / `vi d4`	same idea as nano, different editor

The `echo >` rules she made them figure out

Situation	What happens
File **already exists**	**overwrites** it (does not append)
File **doesn't exist**	**creates** it

Making many files at once

```
touch file1 file2 file3 file4      # several, listed out
touch file{1..10}                  # 10 files in one shot
```

💡 GOOD TO KNOW

Notice what just happened: `{1..10}` is brace expansion — *"it's basically a simple one-line script."* This is your first taste of Class 3.

⚠️ WATCH OUT

The limit of editors: `touch` can create **any number** of files at once. `nano`, `vi`, `vim` — *"with any editor you can create only one file at a time."*

◆ ◆ ◆

07 🔷 Making Directories

```
mkdir mydir                # one directory
mkdir d1 d2 d3             # several at once
mkdir -p d1/d2             # RECURSIVE — creates d1, then d2 inside it
mkdir -p d1/d2 d3          # combine both ideas
```

😄 CLASS MOMENT

How she taught it: *"Check how we can create a recursive directory. You have your laptop. You have ChatGPT. Use it. Tell me the command."* Someone guessed `-r` for recursive — it's `-p` (for *parents*).

✅ LOCK THIS IN

Two separate concepts, don't merge them: 1. **Multiple** directories at once → just list them: `mkdir d1 d2 d3` 2. **Recursive/nested** directories → `mkdir -p parent/child`

◆ ◆ ◆

08 🔷 Getting Around

Command	What it does
`ls`	list files here
`cd <dir>`	go into a directory
`cd ..`	go **up** one level (to the parent)

Command	What it does
`cd` (alone)	jump straight to your **home** directory
`pwd`	**P**rint **W**orking **D**irectory — where am I? (she asked the full form)
`clear`	wipe the console — *"I've used it 20–30 times already today"*

💡 GOOD TO KNOW

PowerShell users: the Windows equivalent of `clear` is `cls`.

◆ ◆ ◆

09 🔷 Copy, Move, Rename, Delete

Delete

Command	Removes
`rm <file>`	a **file** (the class debated file vs directory — it's files)
`rmdir <dir>`	a directory — **only if it's empty**

😄 CLASS MOMENT

The live test: *"We run `rmdir d2`. Will it remove it?"* Room: *"No."* It worked. *"How? It was empty. That's it."*

Copy — `cp`

Two arguments, always:

```
cp <source> <destination>
#    ↑ what to copy    ↑ where to put it
```

Demo: `cp d3 nancy/` → `d3` now exists in both places.

🎯 INTERVIEW ALERT

The trap question she set: the `nancy` folder already contained `d1/`, `file1`, `file2`, `file3`, `file4`. *"If I copy into it — is it going to erase all the files inside, or just copy-paste?"*

✅ **Answer: just copy-paste.** Nothing is erased, no error. Copying **into** a folder doesn't wipe the folder.

Move — `mv`

Same two arguments. The difference:

	Behaviour
`cp`	**copy** and paste — original stays
`mv`	**cut** and paste — original is **gone**

Demo: `mv d3 nancy/` → `d3` disappears from the current folder and appears in `nancy/`.

Rename — also `mv`

🧠 PICTURE THIS

Picture this: on Windows you right-click → Rename. Linux has no "rename" — it uses **move**, because renaming *is* moving a file to a new name in the same folder.

```
mv rename rename1     # "rename" is now called "rename1"
```

🎯 INTERVIEW ALERT

Question to Team C: if the destination name doesn't exist yet, does `mv` create it or fail? → ✅ **It creates it** — that's exactly how renaming works.

◆ ◆ ◆

10 📁 Reading Files

She pasted Docker install instructions into a file to work with — *"Docker is my favourite, so any time I just copy something from 'install docker'."*

```
vi docker.txt         # open the editor
# ...paste content...
:wq                   # save and quit ("write, quit")
cat docker.txt        # print the whole thing
```

Writing without opening an editor

Command	Effect
`echo "this is new file" > f1`	**overwrite** everything in `f1`
`echo "hello world" >> f1`	**append** to the end of `f1`

⚠️ WATCH OUT

Respect the single `>`. *"Whether it's 1000 lines, 1 million, 1 billion or 10 billion lines — it will overwrite in just one enter."* One character difference between adding a line and nuking a file.

Reading big files

Command	Shows
`head <file>`	the **first 10** lines
`tail <file>`	the **last 10** lines
`less <file>`	scroll **page by page** — for big files
`more <file>`	same idea

😄 CLASS MOMENT

The line-counting lesson. She ran `head` and asked how many lines came out. The room counted **8**. The answer was **10**.

"This is the first line. Second. Third… this is a complete one line… eight, ninth, and this is tenth. Got it?"

✅ **The catch: blank lines are lines.** Empty lines count. That's the whole trick.

💡 GOOD TO KNOW

Why `less` / `more` exist: *"If you have a Dockerfile with 1000 lines, you can't just `cat` it — you'd scroll forever and never find anything. Want 10 lines? `head`. Want to read page by page? `less`."*

◆ ◆ ◆

11 📁 Listing & Hidden Files (Her Two Tasks)

Task 1 — list another folder's contents without using `cd`

```
ls nancy          # ✅ just pass the folder name
```

Hidden files

🧠 PICTURE THIS

The rule: put a **dot** at the front of the name and Linux hides it. That's the entire mechanism.

```
touch .env        # hidden (developers will recognise this one)
ls                # → you can't see it
ls -a             # → there it is  (-a = all)
ls -la            # → all files, WITH permissions
```

Task 2 — check the permissions of one specific file in another folder, without `cd`

```
ls -l nancy/data1     # ✅ path + filename
```

⚠️ WATCH OUT

The near-miss: someone answered `ls -l nancy` — that lists *everything* in the folder. *"I can see all files, not the specific data1."* You have to name the file.

◆ ◆ ◆

12 🐧 Processes — Linux's Task Manager

Seeing what's running

Command	What it gives
`ps -A` / `ps aux`	list processes
`top`	live CPU + memory per process

Command	What it gives
`htop`	⭐ *"colourful, and you can even scroll"*
`btop`	mentioned as another option

🧠 PICTURE THIS

Picture this — it's literally Task Manager. *"In Windows you have Task Manager, right? You open it and you can see the browser using this much CPU, ChatGPT using this much, VS Code using this much."*

Her college example: *"If you're an Android developer — Android Studio, and your laptop is down. You can't use anything, you can't even switch off the laptop. So you open Task Manager, right-click Android Studio, and kill it."*

The columns that matter: PID (process ID) · user · %CPU · %memory.

💡 GOOD TO KNOW

Why `htop` over `top`: *"`top` — I don't like to read that kind of content."* `htop` is the human-readable, colourful, scrollable version. She asked the class for it and someone got it.

Killing things

Command	Kills by
`kill <PID>`	the **process ID** number
`pkill <name>`	the **process name** ("process kill")

How to actually do it:

```
htop                # find the PID
kill 1100           # kill that process
pkill ubuntu        # kill by name instead
```

🧠 PICTURE THIS

Same Task Manager analogy: *"In Windows Task Manager, Chrome is running. You right-click that service and exit — your Chrome browser shuts down. Same way: if Docker is running and I `kill <docker PID>`, Docker exits."*

⚠️ WATCH OUT

She refused to run `pkill ubuntu` live — *"otherwise my Ubuntu will crash."* Note the distinction she drew: it kills the **process** named ubuntu, not the **user** named ubuntu. Same word, two different things.

◆ ◆ ◆

13 🔷 `history` — And The Time It Saved A Bank

The command

```
history
```

Everything you've typed since the machine was created, stored in a history file. *"Whatever we played around with from the time we created this instance — it's adding all the commands into one file."*

💡 GOOD TO KNOW

Why you'd actually want it: you ran something system-level, your data vanished, your app crashed, your Ubuntu won't boot. Run `history`, copy the commands, hand them to ChatGPT: *"this is the issue I'm facing, my laptop won't open, I ran these commands"* → it tells you what you broke.

Backstory: how she learned this the hard way

She learned Linux by dual-booting her own laptop. *"Every time I did something my PC would crash. I just took a backup on a pen drive so at least I could get my data back."*

🔥 The banking outage story

This is the best story in the class, so here it is properly.

🔥 HARD MODE

Setting the scene. December, Christmas week. The whole team is on leave — the US folks, the UK folks, even the Chinese colleagues. She *doesn't* take December leave, because — *"I'm a Gujarati"* — she takes her **entire year's leave in one go during Navratri/Garba**, two to two-and-a-half months. So she's one of the very few people at work.

What she did:

- She was experimenting with **networking commands** and the `.bashrc` file.
- 📌 *What `.bashrc` is:* the file where you put environment-specific things — versions, **proxy details**, environment variables.

- Her **Podman** (an alternative to Docker) wasn't working. So she edited `.bashrc` and **redirected traffic from port 8000 to another port**.

What actually happened:

```
1 server touched
  └── in a cluster of 4–5 nodes
        └── handling 20+ worker nodes
              └── each running 10–20 services
                    →  ~150 banking services went DOWN
```

🎤 SAID IN CLASS

"Can you imagine how big an impact I did? And at that time I was like — what the hell did I do? How do I make it up and running?"

The escalation:

- Everyone: *"call the lead, call the lead"* — the Chinese team, the UK team, everyone.
- The **networking lead, 20+ years of experience**: *"who did this?"*
- She owned up: *"Yeah — I was checking the `.bashrc` file, the environment variables, whether my Podman image was working."* → *"So why did you route the traffic?"* → *"Yeah, I did."*
- Two more engineers pulled in from the **Hong Kong** region — during their Christmas. *"They were like, okay, we have Christmas over here, what are we doing?"*

The fix:

✅ LOCK THIS IN

After a **three-hour meeting**, she ran `history`. The senior engineer read through it, immediately spotted the traffic redirect, and fixed it. Application back up.

The damage: ~10 minutes of downtime — one **credit-card module** affected. *"Not much, because there are a lot of servers and backup servers, they'll make it up."* But — *"it's a bank and it has its own credit card, and I made it like this. That was not a good sign."*

The lead's parting advice: *"Nancy, I'll tell you — don't play with networking unless we're on leave. We'll give you other servers, no worries, you want to play, you can play."* 😄

✅ LOCK THIS IN

Her takeaway, and it's the point of telling you: *"If you are not making mistakes, that means you are not learning — in DevOps, at least."*

◆ ◆ ◆

14 🔐 Permissions — `chmod` Properly

The layout

Run `ls -l` and you get something like `-rw-rw-r--`. Break it apart:

```
  -    rw-      rw-      r--
  ↑     ↑        ↑        ↑
type  OWNER   GROUP   OTHERS
```

The first character = what kind of thing it is:

Char	Means
`-`	a **file**
`d`	a **directory**
`l`	a **link**

The numbers

Permission	Number
read	**4**
write	**2**
execute	**1**

🎯 INTERVIEW ALERT

Her bonus question: why 4-2-1? Why not 5-3-1 or 2-3-4?

Her answer: it comes from the **microprocessor** layer. *"Have you ever studied the microprocessor subject? There's an instruction over there — to read one file, when you click one button, it executes four instructions"* (the I/O operations). Hence read = 4, write = 2, execute = 1.

🔍 **Simplified, because there's a cleaner reason worth knowing too:** 4-2-1 are **powers of two** (2^2, 2^1, 2^0), so they're really three **bits** — `rwx` = `111` = 7. That's why every combination adds up to a unique number, which is exactly what a bitmask needs. Both framings land in the same place; the bit-flag one is the version most references give, and it's the one that makes `7 = 4+2+1` feel inevitable rather than arbitrary.

Reading a chmod number

```
chmod 761 data1
```

Slot	Digit	Who	Permission
1st	**7**	**owner** (you, who created it)	4+2+1 = read **+** write **+** execute
2nd	**6**	**group**	4+2 = read **+** write
3rd	**1**	**others**	1 = execute only

Then verify:

```
ls -la data1     # permissions changed from -rw-rw-r-- to the new set
```

🎮 The permission quiz

Asked to	Question	Answer
Team A	read + execute for **owner** only	`500`
Team B	read + write for owner	`600`
Team B	read + execute for **group**	`050`
Team C	read + write + execute for **others**	`007`
Team C	nothing at all for others	`0`
Team A	decode `750`	owner = rwx, group = r-x, others = nothing
everyone	decode `170`	owner = execute only, group = rwx, others = nothing

The two "would you actually do this?" moments

⚠️ **WATCH OUT**

`170` **— would we do this in real life?** *"You have a file. You have only execute permission. And you give a group — maybe thousands of people — permission to execute, read, edit, delete that file. And for others, nothing."*

No. *"Why should we give one group permission to read, write and execute — meaning create, delete and update — a file, while the owner can't?"* It's backwards.

⚠️ WATCH OUT

`007` — **same problem, mirrored.** Giving read/write/execute to *strangers* while keeping nothing for yourself makes no sense either. (This was the point buried under the garbled Hindi stretch of the recording.)

✅ LOCK THIS IN

Rule of thumb: 7 goes to the owner, and only the owner — and even then, only when it's actually needed.

🏆 Which directory gets `777`?

🎯 INTERVIEW ALERT

She asked the class to search for it: *"There is one directory that has 777 permission by default. Guess which one."*

Someone said the **root directory**. *"Hey, are you kidding? If I give 777 to the root directory then anyone can edit any file and anyone can make their own operating system. We never give 777 to root."*

✅ **The answer:** `/tmp`. Because temp data is wiped anyway — *"every time you log in, log out, shut down the server or shut down your laptop and open it again, it removes that temp data."*

📝 Homework #1 — soft link vs hard link

🎯 INTERVIEW ALERT

She flagged it as an interview question and said it will be in the assessment: *"Have you ever created a soft link? A hard link? Do you know the concept? This is a homework."*

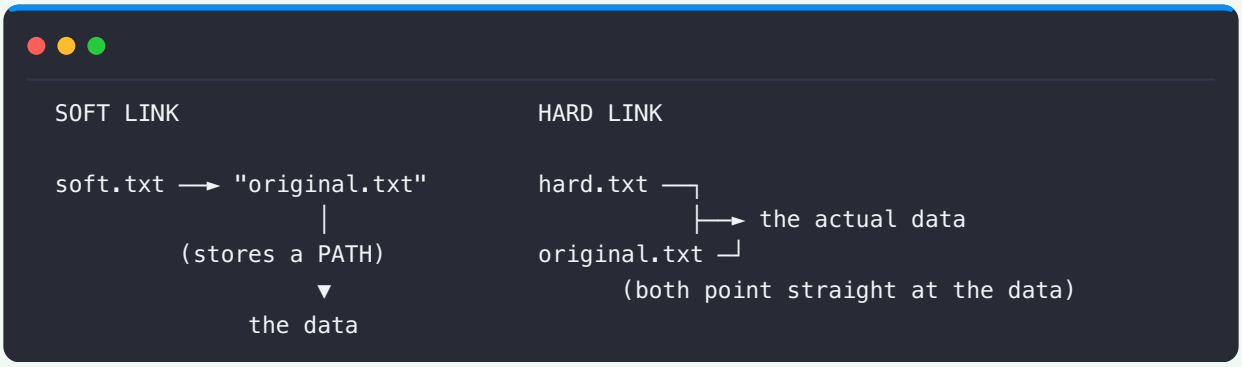
(Context: it's the third file type, `l`, in that first `ls -l` character.)

✅ LOCK THIS IN

The answer — try it yourself first, then check here.

```
echo "hello" > original.txt
ln -s original.txt soft.txt      # -s = SOFT link (symbolic)
ln    original.txt hard.txt      # no flag = HARD link
```

🧠 **Soft link = a shortcut. Hard link = a second name for the same thing.**



Which is why they behave differently when you delete the original:

```
rm original.txt
cat soft.txt      # ❌ No such file or directory   ← the shortcut now points at nothing
cat hard.txt      # ✅ hello                       ← the data is still there
```

	Soft link (`ln -s`)	Hard link (`ln`)
Stores	a **path** to the target	the **same inode** (the data itself)
Delete the original	❌ link breaks ("dangling")	✅ still works
Across different disks/partitions?	✅ yes	❌ no
Can point at a **directory**?	✅ yes	❌ no
Shows in `ls -l` as	`l` and `soft.txt -> original.txt`	a normal file, `-`

🧠 **The everyday version:** a soft link is a **Windows shortcut** — delete the real file and the shortcut is useless. A hard link is **two doors into the same room** — board up one and the room is still there.

✅ **In practice you almost always want `ln -s`.** Hard links are rare; soft links are how `/usr/bin/python` points at `python3.13`.

◆ ◆ ◆

15 🔷 `/` vs `/root` — The Question They Love Asking

🎯 INTERVIEW ALERT

Interview alert, in her own words: *"This is an interview question. If there's an OA round for Linux, or a technical round, or an MCQ — they are going to ask you this."*

Path	What it is
`/`	the **root directory** — the very top of the Linux filesystem
`/root`	the **home directory of the root user**. Root is a **user**.
`/home/ubuntu`	the ubuntu user's directory
`/home/nancy`	*"directory that I created for user Nancy"*

She polled the room first — half said same, half said different, one said neutral. 😄

🧠 PICTURE THIS

The framing that makes it stick: *"Linux is nothing but a simple file and a directory. If you remove that file, there is no Linux. If you remove the `/` directory, your system will crash."*

💡 GOOD TO KNOW

How the question will actually be phrased: not *"are `/` and `/root` the same?"* but *"what do you mean by `/` and `/root` ?"* Answer: `/root` = directory for the root **user**, same as `/home/ubuntu` is for the ubuntu user — and `/` is the filesystem root itself.

◆ ◆ ◆

16 🐧 Tour of the Linux Filesystem

These are the top-level directories in **any** Linux system.

Directory	What lives there
`/bin`	**bin**aries — the actual programs and driver files
`/sbin`	**s**ystem binaries
`/etc`	⭐ **config files** — everything configurable
`/dev`	devices
`/proc`	process files

Directory	What lives there
`/tmp`	temp files (the `777` one)
`/home`	user home directories (`/home/ubuntu` , `/home/nancy`)
`/boot`	boot files
`/lib`	libraries
`/mnt`	mount points
`/srv`	service data (heard as "svc")
`/opt`	**opt**ional configs
`/usr`	user-specific things
`/var`	⭐⭐ **logs** — see below

🧠 How `/bin` actually matters — the microphone story

🧠 PICTURE THIS

Picture this. *"Whenever I speak here, or whenever you hear me from your laptop — how is the microphone working? In hardware there is just one file."*

```
You speak
  → a SYSTEM CALL goes to the KERNEL   ("the kernel is the brain")
    → the kernel calls the DRIVER FILE
      → I/O operations happen
        → sound comes out
```

"Same thing when I type — all my keyboard input goes through another file, and it executes that file, does some I/O operations, and I can see what I type." Those driver/binary files live in `/bin` .

💡 GOOD TO KNOW

What "the kernel" is, since her story hangs on it.

The **kernel** is the one program allowed to touch hardware. Nothing else is — not your browser, not your Python script, not `ls`.

```
your program   "open this file" / "send these bytes"
      │
      │  a SYSTEM CALL  ← the only doorway
      ▼
  ┌──────────┐
  │  KERNEL  │  the only thing with hardware access
  └──────────┘
      ▼
disk · network card · microphone · keyboard
```

✅ **Why it's built this way:** if every program could talk to the disk directly, one buggy program could corrupt everything. Funnelling it all through the kernel means one place enforces the rules.

🧠 **And this is the fact that explains Docker later.** Containers share **the host's kernel** — that's why they're megabytes instead of gigabytes, and why they start in a second. A virtual machine ships a whole second kernel; a container just borrows yours. (Class 6 §4.)

🔍 `/etc` — the config drawer

You've already used it: `cat /etc/os-release`. Two more that matter a lot:

Every user on the system:

```
cat /etc/passwd
```

😄 CLASS MOMENT

"It's shocking, but yes — to check the users you have to give a command like `cat etc password`."

What the fields mean:

```
username : x : UID : GID : description : home path : shell
                                                     └─ e.g. /bin/bash
```

🎯 INTERVIEW ALERT

Interview alert: *"Normally they ask: can you list down all the users inside your system?"* The answer is `cat /etc/passwd`. *"This command is itself weird — you want to list users, so why can't you do `etc/users`? But there's no such command."*

The users you didn't create — `daemon`, `bin`, `sys`, `sync`, `games`, `man`, `lp` and more. These are **system service accounts**, created by default when the OS installs.

💡 GOOD TO KNOW

Watch it happen: *"If you install Docker right now, you'll see the next user is `docker`, right after `ubuntu`."* Installing a service creates its user and group.

Every group on the system:

```
cat /etc/group
```

🗄️ `/var` — where a DevOps engineer actually lives

✅ LOCK THIS IN

Learn this one properly. *"In DevOps, `var` is very important."* All the logs of your complete system live under `/var/log` — and when your application is running on a server, **its** logs go there too.

🎯 *"This is also one of the MCQ questions. If you have an OA round, this will be there — most of the time it will be there."*

The one you can skip

😄 CLASS MOMENT

`/proc` : *"You don't need to know it. It's related to process files. In the past three, four years there is no interviewer who knows about this. There's no chance they'll ask you this question — I'm telling you frankly, don't worry."*

◆ ◆ ◆

17 🔎 `grep` and the Pipe

The pipe `|`

✅ LOCK THIS IN

One-line definition, straight from class: *"It takes the output from the previous command as an input, and processes it to the next command."*

grep in action

```
cat docker.txt | grep docker      # ✅ finds every line containing "docker"
cat docker.txt | grep Docker      # ❌ nothing — capital D doesn't match
cat docker.txt | grep -i docker   # ✅ -i = ignore case → works
```

😄 CLASS MOMENT

She broke it on purpose. After the capital- `D` search returned nothing: *"Why? I mean — you are awake? Okay, that's okay, I'm doing it deliberately to check your consciousness."*

✅ LOCK THIS IN

The lesson: `grep` is case-sensitive by default. `-i` makes it case-insensitive.

◆ ◆ ◆

18 🔗 `systemctl` — Controlling Services

When does something become a "service"? When you install it. *"If I install Docker it installs as a service. Podman, Kubernetes, Jenkins — same."*

What you can do to a service: start · stop · restart · reload · enable · pause · check status.

```
sudo systemctl start docker
sudo systemctl stop docker
sudo systemctl restart docker
sudo systemctl reload docker
sudo systemctl status docker
sudo systemctl enable docker      # start automatically on boot
```

💡 GOOD TO KNOW

It's the same six commands for everything. Swap the last word:

```
sudo systemctl start nginx
sudo systemctl start apache2
sudo systemctl enable apache2
```

"Just replace the word `docker` with `nginx` or `apache`. That's it."

😄 CLASS MOMENT

A student answered *"systemctl is for checking the service status"* — technically true but too narrow, which prompted: *"Sometimes we need to become a good prompt engineer."*

📝 Homework #2 — `journalctl`

🎯 INTERVIEW ALERT

"Second command is `journalctl`. What is it doing? Anyone know? Similar to `systemctl`. This is your homework — check that."

😄 Bonus: she noted that colleagues outside India pronounce it *"journal-CTL"* differently, and one friend of hers says it their own way entirely.

✅ LOCK THIS IN

The answer — `systemctl` controls services, `journalctl` reads their logs.

```
systemctl  →  "start / stop / restart docker"     ← the CONTROL
journalctl →  "why did docker fail to start?"     ← the LOGS
```

The four you'll actually use:

```
journalctl -u docker          # logs for just the docker service
journalctl -u docker -f       # -f = follow, live tail (like tail -f)
journalctl -u docker -n 50    # last 50 lines only
journalctl -p err             # errors only, across everything
```

🧠 **The pairing to remember:** `systemctl start nginx` fails → `journalctl -u nginx -n 50` tells you why. That two-command loop is most of Linux troubleshooting on a real server.

◆ ◆ ◆

19 📝 Homework #3 — User Management (The Real Assignment)

> 📝 **The assignment, in her words:** *"We are going to give you an assignment for user management. This you have to do by yourself."*

Already covered in class: `passwd`, `id`, `groups`.

The specific question she wants you to answer

🎯 INTERVIEW ALERT

There are TWO ways to create a user — `adduser` and `useradd`.

"One is the default one, the standard one — and one is not. Make sure you know which is the default/ standard way and why you cannot use the other one."

The full command set to learn

Command	Does
`adduser`	create a user (one of the two)
`useradd`	create a user (the other one)
`usermod`	**modify** a user — including moving them between groups
`userdel`	delete a user
(group equivalents)	*"same thing we have for groups"* — `groupadd`, `groupmod`, `groupdel`

Why all of these exist: *"If the user is there, then we can create it, modify it, delete it. We can put that user in a group, or in multiple groups. A group might have 10 users, or many. And we can move that user to another group."*

💡 GOOD TO KNOW

"There are only three to four commands here" — it's a small, very learnable set.

✅ LOCK THIS IN

The answer to her `adduser` vs `useradd` question.

	`adduser`	`useradd`
What it is	a **friendly script** wrapped around `useradd`	the **raw, low-level** command
Creates a home directory?	✅ automatically	❌ not unless you pass `-m`
Asks for a password?	✅ prompts you interactively	❌ account is left **locked**
Copies default config files?	✅ from `/etc/skel`	❌ no
Interactive?	✅ asks you questions	❌ silent

```
adduser nancy                              # ✅ done. home dir, password,
everything.

useradd nancy                              # ⚠️ user exists but can't log in
useradd -m -s /bin/bash nancy && passwd nancy    # the equivalent, done by hand
```

🎯 **So the answer she's after: `adduser` is the standard/default way** for a human creating an account, because it does the whole job. **`useradd` isn't "wrong"** — it's the primitive, and it's exactly what you want **inside a script or a Dockerfile**, where you *don't* want something stopping to ask questions.

⚠️ **One portability catch worth knowing:** `adduser` is a Debian/Ubuntu convenience script. On RHEL/CentOS, `adduser` is just a symlink to `useradd` and behaves like the raw command — so on those systems you must pass `-m` yourself.

◆ ◆ ◆

20 🐣 Absolute Basics (for anyone brand-new to Linux)

Command	Does
`Ctrl + C`	cancel the running command
`date`	print the date/time
`clear`	clear the screen (`cls` in PowerShell)
`whoami`	who am I
`hostname`	machine name
`history`	everything you've run

Command	Does
`uptime`	how long the box has been up

◆ ◆ ◆

21 📚 Resources She Promised

Resource	What it is
Advanced Linux command cheat sheet	to be shared with the batch
"DevOps Heroes" repo	the batch repo she'll upload the sheet + resources to
Her personal Linux repo *(private)*	Linux **games** for learning commands, the **Linux Bible** book, cheat sheets, PDFs
Her other repos	networking, disk management, general Linux notes

🧭 Her actual interview-prep method

✅ LOCK THIS IN

"I created a master `.md` file. Whenever I have an interview, for 30 seconds I go through that MD file and my Linux is done. Then I'll go to Docker — maybe half an hour or 15 minutes before the interview. Then Kubernetes. Like that, for every tool I have my own repository."

Commands from that sheet she highlighted:

Command	Purpose
`df` / `df -h`	**disk** usage (`-h` = human readable)
`free` / `free -h` / `free -m` / `free -g`	**memory** usage

💡 GOOD TO KNOW

"These are the only things they even know, most of the time — I'm sure." 😄

🎓 The motivation story

> 😄 **CLASS MOMENT**
>
> **Her brother's story.** He asked for the complete DevOps roadmap she'd prepared, studied it, and **cracked a Linux Foundation interview** — now doing an internship.
>
> The punchline: *"He's not from a computer science background — he's a chemical engineer. He started DevOps three months back. And the funniest fact is he has almost six backlogs. Yeah, still he can do that. Can you imagine? You are a computer science student."*
>
> *"But yeah — it's about passion."*

Class ended with attendance.

◆ ◆ ◆

22 📔 Cheat-Sheet — Every Command From Today

Identity & system info

Command	Does
`whoami`	your username
`who` / `w`	who's logged in
`logname`	login name
`id`	⭐ UID, GID, groups
`echo $USER`	username from the environment
`users`	all logged-in users
`hostname`	machine name
`hostname -s` / `-f` / `-i`	short / full / IP
`hostnamectl`	hostname + hardware info
`cat /etc/hostname`	read it from the file
`cat /etc/os-release`	⭐ full OS details
`uname`	kernel name (partial info)
`lsb_release`	distro info (not the "standard" answer)
`date`	date and time
`uptime`	how long it's been running

Files & directories

Command	Does
`touch f1`	create empty file(s)
`touch f1 f2 f3`	create several
`touch file{1..10}`	create 10 at once
`nano f` / `vi f` / `vim f`	create/edit in an editor (one file)
`:wq`	save & quit in vi/vim
`echo "x" > f`	write (**overwrite**)
`echo "x" >> f`	**append**
`mkdir d`	make a directory
`mkdir d1 d2 d3`	several directories
`mkdir -p d1/d2`	**nested** directories
`ls`	list
`ls <dir>`	list another directory
`ls -a`	include **hidden** (dot) files
`ls -la`	hidden + permissions
`ls -l <dir>/<file>`	permissions of one specific file
`cd d` / `cd ..` / `cd`	into / up / home
`pwd`	where am I
`cp src dst`	**copy**
`mv src dst`	**move** (cut/paste)
`mv old new`	**rename**
`rm f`	delete a file
`rmdir d`	delete an **empty** directory
`cat f`	print whole file
`head f`	first 10 lines
`tail f`	last 10 lines
`less f` / `more f`	page-by-page
`clear`	clear screen

Permissions

Command	Does
`chmod 400 key.pem`	read-only for owner (needed for SSH keys)
`chmod 761 f`	owner rwx, group rw, others x
`chmod 777 /tmp`	the only place that's normal
`ls -l`	see the current permissions

Processes

Command	Does
`ps -A` / `ps aux`	list processes
`top`	live resource usage
`htop`	⭐ colourful, scrollable version
`btop`	another alternative
`kill <PID>`	kill by process ID
`pkill <name>`	kill by name

Search & pipes

Command	Does
`|`	pipe: previous command's output → next command's input
`grep x f`	find lines containing `x`
`grep -i x f`	case-**insensitive** search
`cat f | grep -i x`	the classic combo

Services

Command	Does
`sudo systemctl start|stop|restart|reload|status|enable <svc>`	control a service
`journalctl`	📝 homework — find out what it does

Users & groups

Command	Does
`cat /etc/passwd`	⭐ list all users
`cat /etc/group`	list all groups
`passwd`	change a password
`groups`	which groups am I in
`adduser` / `useradd`	📝 homework — which is standard, and why
`usermod`	modify a user / change groups
`userdel`	delete a user

Packages & disk

Command	Does
`sudo apt update`	refresh package lists
`sudo apt upgrade -y`	install updates, no prompts
`df` / `df -h`	disk usage
`free -h` / `-m` / `-g`	memory usage
`ifconfig`	network config (needs `net-tools` installed)

AWS / SSH

Command	Does
`ssh -i key.pem ubuntu@<ip>`	log into your EC2
`chmod 400 key.pem`	required before SSH will accept the key

◆ ◆ ◆

23 📝 Homework Round-Up

#	Task	Why it matters
1	**Soft link vs hard link**	🎯 interview question; she said it'll be in the assessment
2	**What does `journalctl` do?**	companion to `systemctl`

#	Task	Why it matters
3	**User management assignment**	`adduser` vs `useradd` — which is standard and **why** the other isn't; plus `usermod`, `userdel`, and the group equivalents
4	Practice on your own EC2	use your student ID for the 6-month free tier

◆ ◆ ◆

24 🗒️ Notes on the Transcript Itself

- **Instructor name:** the transcript header credits *"Ritesh Prajapati"* — that's the **developer of the Scaler++ Chrome extension** that downloaded the file, not the teacher. The instructor is **Nensi Ravaliya** (@Nency-Ravaliya), a **DevSecOps Engineer at Standard Chartered** — which is the bank in the outage story in §13. She's addressed as *"Nancy"* by her networking lead there, names her demo directory `nancy`, and says *"forward slash and Nancy means directory that I created for user Nancy."* Class 3 (`"this is my name is Nancy"`) and Class 8 (a terminal path reading `/Users/Nensi Ravaliya/...`) confirm it.
- The recording **opens with Norwegian subtitle credits** (*"Undertekster av Amara.org-gemenskapen"*) repeated several times — a dubbing/translation artifact from the transcription tool, not speech.
- A stretch of the **chmod quiz** (around the `007` / `170` questions) comes through as garbled Hindi. The recoverable point — *"why would you give read/write/execute to other people and not to yourself, if it's your own file?"* — is folded into §14.
- Garbled terms corrected from context: **"CH more/CH mode"** → `chmod`, **"Uber / Uber to / open to"** → `ubuntu`, **"pan file / spam file"** → **PEM file**, **"New fetch"** → `neofetch`, **"Unim"** → `uname`, **"lsv-release"** → `lsb_release`, **"mtdir"** → `mkdir`, **"nancy folder / NIMSY folder"** → the `nancy` **directory**, **"card etc password"** → `cat /etc/passwd`, **"demon"** → `daemon`, **"H2 laptop"** → **an old laptop she dual-booted**, **"Garganta Navratri"** → **Garba/Navratri**, **"svc"** → `/srv`, **"CLS"** → `cls`.
- The `hostname -i` line is her genuinely discovering a flag live: *"This is a new command, even I don't know."*